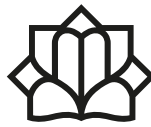


بسمه تعالی



دانشگاه کاشان
University of Kashan

دانشگاه کاشان

دانشکده مهندسی برق و کامپیوتر

گروه مهندسی کامپیوتر

گزارش پروژه کارشناسی مهندسی کامپیوتر

سامانه هوشمند مدیریت
حمل و نقل بار جاده‌ای
لجستیک

نسخه موبایل، وب پیش‌رونده و سرویس‌دهنده

استاد راهنما

دکتر سید مرتضی بابامیر

نقش	نام	شماره دانشجویی
فرانت‌اند	حانیه نباتی	40121160028
بک‌اند و پایگاه داده	مهیار عامری	40121160021

کارشناسی مهندسی کامپیوتر · شهریور ۱۴۰۵

شرح طراحی، پیاده‌سازی و استقرار سامانه

چکیده

سامانه لجستیک یک سکوی نرم افزاری برای مدیریت حمل و نقل بار جاده‌ای است که دو نقش اصلی اکوسیستم حمل را به یکدیگر متصل می‌کند: **راننده و متصدی حمل و نقل** (اپراتور). متصدی بار را با مشخصات کالا، نوع ماشین، وزن، قیمت، مبدأ و مقصد ثبت می‌کند؛ راننده بارهای متناسب با ماشین خود را می‌بیند، در صورت نزدیکی جغرافیایی پیشنهاد نزدیک دریافت می‌کند، بار را می‌پذیرد و تا مبدأ و سپس مقصد مسیریابی می‌شود. تکمیل مأموریت نیازمند تأیید هر دو طرف است تا وضعیت بار از تخصیص به تحویل قطعی تغییر کند.

سمت کاربر با چارچوب فلاتر پیاده‌سازی شده و علاوه بر اندروید، به صورت وب پیش‌رونده نیز قابل نصب است. رابط کاربری دوزبانه (فارسی و انگلیسی)، راست‌به‌چپ، دارای حالت تاریک، ورود زیست‌سنجی و نشست امن مبتنی بر ژتون دسترسی است. نقشه و مسیریابی خیابانی با خدمات نشان یکپارچه شده است.

سمت سرور با جنگو و لایه رست فریم‌ورک نوشته شده، احراز هویت را با ژتون دسترسی انجام می‌دهد و داده‌ها را از طریق نگاشت شیء-رابطه‌ای جنگو روی پایگاه داده پیش‌فرض اس‌کیولایت ذخیره می‌کند. ارتباط کلاینت و سرور بر اساس قرارداد وب‌سرویس روی میزبان استقرار لیارا انجام می‌شود.

واژگان کلیدی: حمل و نقل جاده‌ای، لجستیک، فلاتر، جنگو، DRF، SQLite، JWT، وب پیش‌رونده، مسیریابی نشان.

فهرست مطالب

۲	چکیده
۴	۱ مقدمه
۶	۲ تحلیل نیازمندی‌ها
۹	۳ معماری سامانه
۱۲	۴ پیاده‌سازی رابط کاربری
۱۵	۵ پیاده‌سازی سرویس‌دهنده
۱۷	۶ پایگاه داده
۲۰	۷ وب‌سرویس‌ها و امنیت
۲۳	۸ استقرار و آزمون
۲۵	۹ نتیجه‌گیری
۲۷	چکیده انگلیسی

فصل ۱ مقدمه

۱.۱ بیان مسئله

حمل و نقل بار جاده‌ای بخش عمده‌ای از جابه‌جایی کالا در ایران را تشکیل می‌دهد. در عمل، اتصال میان صاحب‌بار یا متصدی حمل با راننده مناسب هنوز اغلب از مسیرهای غیرساخت‌یافته انجام می‌شود: تماس تلفنی، گروه‌های پیام‌رسان، یا واسطه‌های سنتی. این روش‌ها چند مشکل مشخص ایجاد می‌کنند:

- **عدم تطابق ماشین و بار:** بار یخچالی نباید به کفی معمولی پیشنهاد شود؛ نبود فیلتر نوع بارگیر باعث اتلاف وقت دو طرف است.
 - **شفاف نبودن وضعیت:** راننده و متصدی تصویر واحدی از این‌که بار «باز»، «تخصیص‌یافته» یا «تحويل‌شده» است ندارند.
 - **ضعف موقعیت مکانی:** پیشنهاد بار نزدیک به محل فعلی راننده، بدون موقعیت واقعی GPS، عملاً تصادفی است.
 - **مسیریابی غیردقیق:** خط مستقیم روی نقشه جایگزین مسیر خیابان و جاده می‌شود و برای راننده کاربردی نیست.
 - **تأیید یک‌طرفه اتمام کار:** اگر فقط یک طرف مأموریت را بسته اعلام کند، اختلاف حساب و وضعیت بار باقی می‌ماند.
- سامانه لجستیک برای پاسخ به همین خلأ طراحی شده است: یک کاربرد دوطرفه با قرارداد مشخص داده، نقش‌های جدا، چرخه وضعیت بار، و مسیریابی واقعی.

۲.۱ هدف پروژه

هدف کلی، طراحی و پیاده‌سازی یک سامانه سرتاسری (کلاینت + سرویس‌دهنده + پایگاه داده) است که:

۱. ثبت، ویرایش، حذف و مشاهده بار را برای متصدی ممکن کند؛
۲. فهرست بارهای باز را با فیلتر استان مبدأ/مقصد و بازه قیمت در اختیار راننده بگذارد؛
۳. پذیرش بار را به راننده‌ای محدود کند که نوع ماشین او با بار سازگار باشد؛
۴. مأموریت‌های تخصیص‌یافته را جدا از بارهای باز نگه دارد؛
۵. مسیر خیابانی از موقعیت راننده تا مبدأ بار و سپس تا مقصد را نمایش دهد؛
۶. اتمام کار را منوط به تأیید راننده و متصدی کند؛
۷. روی موبایل و به صورت وب پیش‌رونده قابل استفاده باشد.

۳.۱ تقسیم کار دانشجویان

پروژه به صورت تیمی و با مرز مشخص لایه‌ها انجام شده است. جدول ۱.۱ این تقسیم را نشان می‌دهد.

جدول ۱.۱: تقسیم مسئولیت در پروژه

مهیار عامری 40121160021 بک‌اند و پایگاه داده سرویس‌دهنده جنگو وب‌سرویس و لایه رست مدل دامنه و پایگاه داده احراز هویت با ژتون دسترسی	حانیه نباتی 40121160028 فرانت‌اند رابط کاربری موبایل و وب معماری کلاینت فلاتر نقشه و وب پیش‌رونده اتصال به وب‌سرویس و آزمون
---	---

استاد راهنمای پروژه جناب آقای دکتر سید مرتضی بابامیر، استاد مهندسی نرم‌افزار دانشگاه کاشان هستند.

۴.۱ قلمرو و محدودیت‌ها

قلمرو سامانه، حمل بار جاده‌ای داخلی با دو نقش راننده و متصدی است. پرداخت برخط، بورس بار سراسری، و ناوگان چندراننده‌ای در نسخه جاری پیاده‌سازی نشده است. اعلان فشاری واقعی (FCM) در کلاینت به صورت زیرساخت آماده مانده و هنوز به نقطه پایانی سرور وصل نیست. پایگاه پیش‌فرض جنگو (SQLite3) برای توسعه و استقرار آموزشی انتخاب شده و برای بار بسیار زیاد تولید می‌توان آن را با PostgreSQL جایگزین کرد، بدون تغییر منطق ORM.

۵.۱ ساختار گزارش

فصل دوم نیازمندی‌ها و مدل کسب‌وکار را بیان می‌کند. فصل سوم معماری کلی را نشان می‌دهد. فصل چهارم پیاده‌سازی فرانت‌اند را شرح می‌دهد. فصل پنجم بک‌اند جنگو و DRF را توضیح می‌دهد. فصل ششم پایگاه داده و نگاشت شیء-رابطه‌ای است. فصل هفتم قرارداد API و امنیت است. فصل هشتم استقرار و آزمون را می‌آورد و فصل نهم جمع‌بندی است.

فصل ۲ تحلیل نیازمندی‌ها و مدل کسب‌وکار

۱.۲ بازیگران سیستم

دو بازیگر انسانی اصلی وجود دارد. نقش در زمان ثبت‌نام انتخاب می‌شود و سرور آن را با پرچم‌های `is_driver` و `is_operator` نگه می‌دارد. کلاینت هنگام ورود بررسی می‌کند که نقش انتخاب‌شده در صفحه ورود با نقش ذخیره‌شده در حساب یکی باشد؛ در غیر این صورت ورود رد می‌شود.

راننده: صاحب یا بهره‌بردار یک دستگاه حمل با پلاک ایرانی، نوع بارگیر (ماشین)، مدل و استان فعالیت. او بار باز می‌بیند، می‌پذیرد، مسیریابی می‌کند و تحویل را تأیید می‌کند.

متصدی حمل‌ونقل (اپراتور): ثبت‌کننده و مدیر بار. بار ایجاد می‌کند، جزئیات را ویرایش می‌کند، در صورت نیاز حذف می‌کند و پس از انجام کار، تکمیل را از سمت خود تأیید می‌کند.

یک کاربر نمی‌تواند همزمان از یک حساب واحد هر دو نقش را در کلاینت فعال کند؛ تمایز نقش در قرارداد API صریح است (`user_type` برابر `driver` یا `operator`).

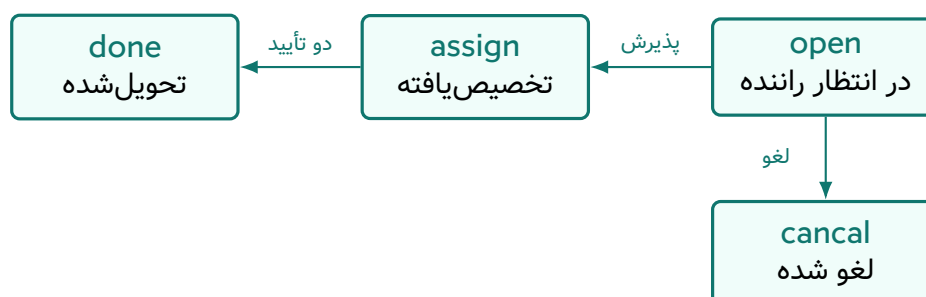
۲.۲ چرخه عمر بار

بار در سرور با فیلد وضعیت نگهداری می‌شود. مقادیر خام دامنه به زبان انگلیسی هستند و کلاینت آن‌ها را به برچسب فارسی نگاشت می‌کند (جدول ۱.۲).

جدول ۱.۲: وضعیت بار در دامنه و نمایش کاربر

کد سرور	معنی	نمایش در اپلیکیشن
open	بار آزاد و قابل پذیرش	در انتظار راننده
assign	به راننده تخصیص یافته	تخصیص یافته
done	تحویل قطعی پس از دو تأیید	تحویل شده
cancel	لغو (املائی قرارداد سرور)	لغو شده

گذارهای مجاز از دید کسب‌وکار در شکل ۱.۲ آمده است.



شکل ۱.۲: گذار وضعیت بار از راست به چپ: ایجاد، تخصیص، تحویل یا لغو

به بیان کوتاه:

- ایجاد بار ← **open**
 - پذیرش راننده ← **assign** و ساخت رکورد تخصیص
 - تأیید راننده و متصدی ← **done**
 - لغو بار باز توسط متصدی ← **cancel**
- نکته طراحی مهم این است که کلاینت برای اعلام اتمام کار، وضعیت را با **PATCH** عوض نمی‌کند؛ بلکه به نقطه پایانی صریح **complete/** با روش **POST** و بدون بدنه درخواست می‌زند. این کار از دور زدن ماشین وضعیت جلوگیری می‌کند.

۳.۲ نیازمندی‌های کارکردی

۱.۳.۲ حساب کاربری

ورود با شماره موبایل ایران و رمز عبور؛ ثبت نام با نام، نام خانوادگی، تأیید رمز، و برای راننده انتخاب نوع ماشین و استان؛ مشاهده و ویرایش پروفایل؛ تغییر رمز با رمز قدیم؛ خروج و ابطال نشست محلی.

۲.۳.۲ مدیریت بار (متصدی)

ایجاد بار با عنوان، توضیحات، کالا (**product**)، نوع ماشین (**machine**)، وزن، قیمت، استان و نشانی مبدأ و مقصد؛ فهرست بارهای متعلق به متصدی احراز هویت شده؛ جزئیات بار و راننده تخصیص یافته در صورت وجود؛ ویرایش و حذف؛ تأیید تکمیل.

۳.۳.۲ عملیات راننده

فهرست صفحه بندی شده بارهای باز با فیلتر استان مبدأ، استان مقصد و بازه قیمت؛ پیشنهاد بار نزدیک در شعاع ۲۵ کیلومتر بر اساس GPS واقعی؛ اجبار تکمیل اطلاعات خودرو قبل از پذیرش؛ پذیرش بار؛ فهرست مأموریت‌های تخصیص یافته؛ مسیریابی تا مبدأ و مقصد؛ تأیید تکمیل از سمت راننده.

۴.۳.۲ داده مرجع

فهرست کالاها، انواع ماشین و استان‌ها از سرور خوانده می‌شود تا شناسه‌های خارجی در بار و پروفایل راننده یکسان بماند.

۴.۲ نیازمندی‌های غیرکارکردی

- امنیت: ژتون دسترسی در ذخیره ساز امن دستگاه؛ ارسال در سرآیند **Authorization: Bearer**؛ مهلت بی‌فعالیتی ۱۵ دقیقه برای اجبار ورود مجدد.
- بومی سازی: فارسی پیش فرض با چینش راست به چپ و فونت وزیر؛ انگلیسی با فونت Inter؛ تقویم و ارقام در متن‌های کاربری فارسی.
- دسترسی پذیری ظاهری: حالت روشن و تاریک با رنگ سازمانی سبز تهرانی (#0F766E).
- چندسکویی: یک کدبیس فلاتر برای اندروید و وب؛ نسخه PWA قابل نصب با کارکرد آفلاین محدود پوسته برنامه.

- **آزمون‌پذیری:** قرارداد OpenAPI در آزمون‌های واحد کلاینت قفل شده است تا تغییر ناخواسته فیلدها مشخص شود.

۵.۲ موجودیت‌های دامنه

چهار موجودیت محوری سامانه عبارت‌اند از کاربر، بار (Bar)، تخصیص (Assign) و داده‌های مرجع (کالا، ماشین، استان). بار به کالا و ماشین و دو استان مبدأ/مقصد وصل است. تخصیص، بار را به راننده پیوند می‌دهد و دو پرچم تأیید را نگه می‌دارد. جزئیات فیلدها در فصل پایگاه داده آمده است.

۶.۲ سناریوی اصلی موفقیت

۱. متصدی وارد می‌شود و بار «حمل سیمان تهران به اصفهان» را با ماشین کفی و وزن مشخص ثبت می‌کند. وضعیت open است.
۲. راننده کفی وارد می‌شود، در صورت نیاز پلاک و مدل را کامل می‌کند، بار را در فهرست یا در بارهای نزدیک می‌بیند و می‌پذیرد.
۳. سرور تخصیص می‌سازد و وضعیت assign می‌شود. راننده مأموریت را در صفحه مأموریت‌ها می‌بیند.
۴. راننده مسیر را تا مبدأ بار و سپس مقصد روی نقشه نشان دنبال می‌کند.
۵. پس از تخلیه، راننده تکمیل را می‌زند (confirm_driver). متصدی نیز تکمیل را تأیید می‌کند (confirm_operator). بار done می‌شود.

فصل ۳ معماری کلی سامانه

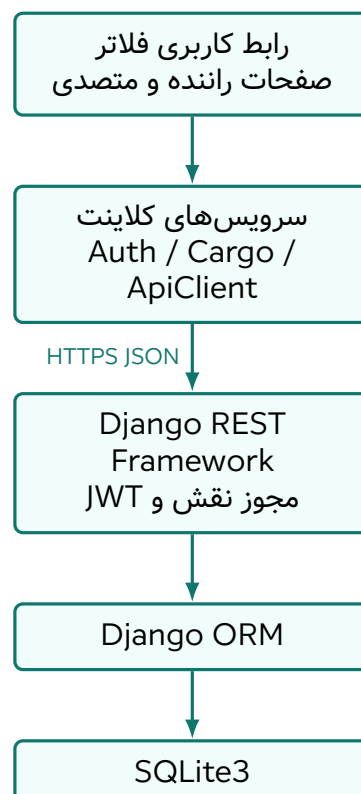
۱.۳ سبک معماری

سامانه در الگوی **کلاینت-سرور** با واسط REST پیاده‌سازی شده است. کلاینت هیچ منطق پایداری بلندمدت کسب‌وکار را در خود نگه نمی‌دارد؛ منبع حقیقت، پایگاه داده جنگو است. کلاینت برای تجربه کاربری، ژتون و تصویر فشرده کاربر را به صورت امن ذخیره می‌کند و در صورت قطع شبکه می‌تواند پوسته PWA را از کش نمایش دهد، اما عملیات بار همیشه از سرور می‌آید.

یک حالت توسعه با داده ساختگی (**USE MOCK API**) در کلاینت باقی مانده تا رابط کاربری بدون سرور قابل نمایش باشد. در نسخه عملیاتی این پرچم خاموش است و آدرس پایه برابر میزبان لیارا است.

۲.۳ نمای منطقی لایه‌ها

شکل ۱.۳ جریان اصلی را نشان می‌دهد.



شکل ۱.۳: لایه‌های منطقی سامانه لجستیک

۳.۳ مؤلفه‌های سمت کاربر

کد کلاینت بر اساس ویژگی (feature-first) سازمان یافته است:

- **lib/features/auth**: ورود و ثبت‌نام

- `lib/features/driver`: خانه، بارها، نزدیک، مأموریت، مسیر، خودرو
 - `lib/features/coordinator`: خانه، فهرست و جزئیات بار، ایجاد و ویرایش
 - `lib/features/shared`: پروفایل، تنظیمات، راهنما، پشتیبانی، درباره
 - `lib/services`: نمای یکپارچه روی HTTP (احراز هویت، بار، داده مرجع، موقعیت، نقشه)
 - `lib/models`: مدل‌های دامنه کلاینت مستقل از جزئیات جنگو
 - `lib/services/transport_api_mapper.dart`: ترجمه قرارداد سرور به مدل کلاینت
- مسیریابی صفحات با `go_router` و مدیریت وضعیت سراسری با `provider` انجام می‌شود. `AuthService` از `ChangeNotifier` ارث می‌برد و روتر با `refreshListenable` به آن وصل است؛ بنابراین تغییر ورود/خروج بلافاصله مسیر را عوض می‌کند.

۴.۳ مؤلفه‌های سمت سرور

- سمت سرور یک پروژه جنگو است که منطقاً به چند بخش دامنه تقسیم می‌شود:
- **حساب‌ها (accounts)**: ثبت‌نام، ورود، پروفایل، تغییر رمز، صدور ژتون.
 - **بار متصدی (operator/bars)**: CRUD بار متعلق به کاربر اپراتور و تکمیل سمت متصدی.
 - **بار راننده (driver/bars)**: فهرست بار باز، پذیرش، مأموریت‌های تخصیص‌یافته، تکمیل سمت راننده.
 - **مرجع (ostans, machines, products)**: جداول کمکی برای انتخاب‌های فرم.
- چارچوب DRF مسئولیت سریالایز، اعتبارسنجی ورودی، صفحه‌بندی و کد وضعیت HTTP را بر عهده دارد. مجوزها بر اساس نقش، دسترسی به ویوهای راننده یا متصدی را محدود می‌کنند.

۵.۳ ارتباط و پیکربندی

آدرس پایه در کلاینت از متغیر زمان ساخت `API_BASE_URL` خوانده می‌شود. مقدار پیش‌فرض این آدرس در استقرار آموزشی به صورت زیر است:

```
https://tran-develoop.liara.run
```

مسیرها در کلاس `ApiConfig` متمرکز شده‌اند تا یک منبع برای قرارداد وجود داشته باشد. همه درخواست‌های نیازمند هویت، سرآیند `Bearer` می‌گیرند.

نقشه نشان از کلاینت و در صورت وجود، از مسیرهای واسط اختیاری سرور `/transport/neshan/` قابل فراخوانی است. مسیرهای نشان جزء قرارداد اصلی حمل‌ونقل نیستند و برای مخفی کردن کلید در سمت سرور پیش‌بینی شده‌اند.

۶.۳ تصمیم‌های کلیدی معماری

۱. یک کدبیس کلاینت: فلاتر اجازه داد منطق کسب‌وکار کلاینت برای اندروید و وب تکرار نشود.
۲. REST به جای اتصال مستقیم به پایگاه: کلاینت هرگز به SQLite سرور وصل نمی‌شود؛ این جداسازی امنیت و استقرار مستقل را ممکن می‌کند.

۳. **JWT بدون نشست سرور:** مقیاس‌پذیری و استقرار روی لیارا ساده‌تر است. نقطه تازه‌سازی ژتون در قرارداد فعلی OpenAPI وجود ندارد؛ بنابراین کلاینت در خطای ۴۰۱ کاربر را به ورود برمی‌گرداند.
۴. **منبع حقیقت وضعیت در سرور:** کلاینت برچسب فارسی نشان می‌دهد اما کد انگلیسی را ذخیره و ارسال می‌کند.
۵. **تأیید دوگانه تکمیل:** از به‌روزرسانی آزادانه فیلد وضعیت توسط کلاینت جلوگیری شده است.

فصل ۴ طراحی و پیاده‌سازی رابط کاربری

این فصل کار انجام‌شده توسط **حانیه نباتی** را شرح می‌دهد: کاربرد فلاتر با نام بسته **legestic**، نسخه ۱.۰.۰.

۱.۴ پشته فناوری کلاینت

جدول ۱.۴: کتابخانه‌های اصلی سمت کاربر

فناوری	نقش
Flutter 3.41 / Dart 3.11	چارچوب چندسکویی رابط کاربری
go_router	مسیرها، شل راننده/متصدی، تغییر مسیر بر اساس نشست
provider	تزریق AuthService و SettingsService
http	کلاینت REST
flutter_secure_storage	ژتون JWT و رونوشت کاربر
shared_preferences	زبان، تم، زمان پس‌زمینه شدن برنامه
local_auth	ورود با اثر انگشت، چهره، پین یا الگو
geolocator	موقعیت واقعی راننده
Neshan MapLibre	نقشه و مسیر خیابانی؛ در وب پل جاوااسکریپت
flutter_map	نقشه پشتیبان در صورت نبود SDK بومی

فونت‌ها به صورت محلی بسته‌بندی شده‌اند (وزیر، Inter، Nasalization) تا برنامه به خدمات فونت اینترنتی وابسته نباشد.

۲.۴ پوسته برنامه و مسیریابی

نقطه ورود **main.dart** تنظیمات را بارگذاری می‌کند و **LogisticsApp** را می‌سازد. روتر با مسیر اولیه / صفحه آغازین را نشان می‌دهد. قواعد تغییر مسیر:

- کاربر مهمان به جز ورود و ثبت‌نام به **/login** فرستاده می‌شود.
- کاربر واردشده اگر به ورود/ثبت‌نام برود، بر اساس نقش به **/driver** یا **/coordinator** می‌رود.
- صفحات اصلی هر نقش داخل **ShellRoute** هستند تا نوار پایین و کشو حفظ شود.

برای راننده مسیرهای شاخص عبارت‌اند از خانه، فهرست بار، مأموریت‌ها، بارهای نزدیک، پروفایل، خودرو، جزئیات بار و صفحه مسیر. برای متصدی: خانه، فهرست بار، ایجاد، ویرایش و جزئیات. صفحات تنظیمات، راهنما، پشتیبانی، درباره و تغییر رمز برای هر دو نقش تکرار شده‌اند تا زمینه نقش در کشو از دست نرود.

۳.۴ احراز هویت در کلاینت

AuthService نمای واحد ورود، ثبت نام، پروفایل، خودرو و تغییر رمز است.

- ورود بدنه phone_number و password را می فرستد، از پاسخ tokens.access (و در صورت وجود refresh) را در ذخیره ساز امن می نویسد و شیء کاربر را می سازد.
 - ثبت نام نام کامل را به last_name/first_name می شکند، user_type را می فرستد و چون پاسخ ثبت نام ژتون ندارد، بلافاصله ورود را صدا می زند.
 - بازیابی نشست در اسپلش: اگر از آخرین بار پس زمینه شدن بیش از ۱۵ دقیقه گذشته باشد، ژتون پاک و ورود مجدد خواسته می شود؛ وگرنه پروفایل از /api/accounts/profile/ تازه می شود.
 - ورود زیست سنجی روی وب غیرفعال است و روی اندروید اعتبار ذخیره شده را پس از موفقیت local_auth به همان مسیر ورود معمولی می دهد.
- شماره تلفن قبل از ارسال با تابع نرمال سازی ایران به قالب استاندارد تبدیل می شود.

۴.۴ لایه دسترسی به داده در کلاینت

ApiClient متدهای GET، POST، PUT، PATCH و DELETE را با کدگذاری UTF-8 و تفسیر خطاهای HTTP به پیام فارسی/انگلیسی فراهم می کند. CargoService فهرست، صفحه بندی، پذیرش، تکمیل، ایجاد و ویرایش بار را کپسوله می کند. ReferenceDataService کالا، ماشین و استان را می خواند.

ترجمه پاسخ سرور در TransportApiMapper متمرکز است؛ از جمله:

- ساخت برچسب مبدأ/مقصد از نشانی یا نام استان؛
 - خواندن وزن از فیلد weight یا در نسخه های قدیمی تر از پسوند متن توضیحات؛
 - استخراج assign_info برای نام راننده و پرچم های تأیید؛
 - ساخت بدنه ایجاد/ویرایش بار مطابق سریالایزر BarCreateUpdate.
- این جداسازی باعث می شود صفحات فلاتر با مدل Cargo کار کنند و به اسامی فیلد جنگو وابسته نباشند.

۵.۴ صفحات راننده

خانه خلاصه وضعیت و میانبر عملیات است. بارها فهرست باز را با فیلتر استان و قیمت نشان می دهد. بارهای نزدیک موقعیت فعلی را می گیرد، بارهایی که مبدأشان در شعاع ۲۵ کیلومتر است مرتب می کند و فاصله را نمایش می دهد. جزئیات بار اطلاعات کامل و دکمه پذیرش دارد؛ اگر پلاک، مدل یا نوع ماشین ناقص باشد کاربر به فرم خودرو هدایت می شود. مأموریت ها از /api/driver/bars/assigned/ با پیمایش همه صفحات ساخته می شود. مسیریابی با SDK نشان مسیر خیابانی، مانور بعدی، دکمه بازگشت به مسیر و نمای جهت دار راننده را نشان می دهد. پلاک ایران با ویجت اختصاصی (ایران، حرف و ارقام) ویرایش می شود.

۶.۴ صفحات متصدی

خانه آمار بارهای متصدی را نشان می‌دهد. فهرست بارها وضعیت هر رکورد را با تراشه رنگی مشخص می‌کند. فرم ایجاد، کالا و ماشین را از داده مرجع انتخاب می‌کند، وزن و قیمت می‌گیرد و نشانی مبدأ/مقصد را می‌فرستد. برآورد قیمت در کلاینت یک رابطه ساده بر اساس وزن و طول متن مسیر است و جایگزین موتور قیمت‌گذاری سرور نیست. جزئیات بار راننده تخصیص یافته و دکمه‌های ویرایش، حذف و تأیید تکمیل را دارد.

۷.۴ نقشه و موقعیت

موقعیت با `geolocator` خوانده می‌شود و دسترسی در گفتگوی اختصاصی از کاربر خواسته می‌شود. روی اندروید نقشه با `Neshan MapLibre` و کانال سکویی رندر می‌شود؛ روی وب `neshan_map_bridge.js` همان تجربه را به `MapboxGL` نشان وصل می‌کند. اگر مسیر کامل از خدمت نشان نیاید، کلاینت مسیر تقریبی را به عنوان حالت تنزل یافته نشان می‌دهد نه این که خطای خام بدهد.

۸.۴ وب پیش‌رونده

پوشه `web/` شامل `manifest.json` (نام فارسی، راست به چپ، نمایش `standalone`، رنگ زمینه سازمانی) و سرویس ورکر اختصاصی `pwa_service_worker.js` است. پوسته برنامه در کش نسخه بندی شده ذخیره می‌شود تا به روزرسانی با تغییر نام کش به دست کاربر برسد. بارگذار اولیه قبل از اولین فریم فلاتر نمایش داده می‌شود. فایل `htaccess`. بازنویسی مسیرهای تک صفحه ای و جلوگیری از کش شدن `index.html` را تنظیم می‌کند. خروجی آماده استقرار در `dist/logistics-` `pwa-build` قرار می‌گیرد.

۹.۴ کیفیت و آزمون کلاینت

آزمون‌های واحد قرارداد فیلد وزن، پذیرش بار، تکمیل دو نقش، جهش‌های متصدی، آمادگی خودرو برای پذیرش، و واکنش گرایی ورود و کشو را پوشش می‌دهند. این آزمون‌ها تضمین می‌کنند کلاینت برخلاف قرارداد سرور `PATCH` روی وضعیت `done` نزند و صفحه بندی مأموریت‌ها را کامل بخواند.

فصل ۵ طراحی و پیاده‌سازی سرویس‌دهنده

این فصل کار انجام‌شده توسط **مهیار عامری** را شرح می‌دهد: خدمت پشتیبان با Django، لایه REST با Django REST Framework، و منطق دامنه روی ORM جنگو.

۱.۵ انتخاب فناوری

جنگو برای این پروژه به سه دلیل انتخاب شد. نخست، ORM بالغ آن مدل‌های حمل‌ونقل را بدون SQL دستی نگه می‌دارد. دوم، DRF سریالایزر، مجوز، صفحه‌بندی و مستندسازی OpenAPI را یکجا می‌دهد. سوم، استقرار روی سکوی لیارا برای کاربرد آموزشی با کمترین پیکربندی عملی است. پایگاه پیش‌فرض SQLite3 با تنظیمات استاندارد جنگو برای حجم فعلی داده کافی است و مهاجرت به موتور دیگر تنها با تغییر تنظیمات اتصال انجام می‌شود.

۲.۵ سازمان پروژه جنگو

پروژه از الگوی چنداپلیکیشن جنگو پیروی می‌کند. اپلیکیشن حساب‌ها کاربر سفارشی (یا پروفایل متصل به کاربر) را با شماره موبایل به‌عنوان شناسه ورود مدیریت می‌کند. اپلیکیشن حمل، مدل بار و تخصیص را دارد. اپلیکیشن مرجع، کالا، ماشین و استان را جدا نگه می‌دارد تا فهرست‌های فرم در کلاینت پایدار بماند.

تنظیمات اصلی:

- `INSTALLED_APPS` شامل `rest_framework` و بسته ژتون ساده (Simple JWT) است.
- احراز هویت پیش‌فرض DRF روی `JWTAuthentication` تنظیم شده است.
- صفحه‌بندی از نوع شماره صفحه است؛ پاسخ شامل `total_pages`، `current_page`، `results` و `next` است تا کلاینت بتواند همهٔ مأموریت‌ها را حلقه کند.
- `LANGUAGE_CODE` و منطقه متناسب با ایران است؛ خودِ کد وضعیت بار انگلیسی می‌ماند تا قرارداد پایدار بماند.

۳.۵ احراز هویت و صدور ژتون

ورود `POST /api/accounts/login` شماره و رمز را می‌گیرد، کاربر را با `authenticate` جنگو بررسی می‌کند و در صورت موفقیت دو چیز برمی‌گرداند: شیء کاربر (شامل `is_driver` و `is_operator`) و ژتون‌ها. ثبت‌نام `POST /api/accounts/register/` کاربر را می‌سازد، رمز را با `set_password` درهم می‌کند و برای راننده `machine_id` و `ostan_id` را روی پروفایل می‌نویسد. ثبت‌نام ژتون نمی‌دهد تا ساخت حساب از شروع نشست جدا بماند.

پروفایل `GET/ PUT /api/accounts/profile/` فیلدهای نام، کد ملی، و برای راننده پلاک (`pelak`)، مدل، ماشین و استان را به‌روز می‌کند. تغییر رمز `old_password` را با `check_password` می‌سنجد و سپس رمز جدید را ذخیره می‌کند. خروج در سمت سرور می‌تواند ژتون تازه‌سازی را در فهرست سیاه بگذارد؛ کلاینت در هر حال ذخیره‌ساز محلی را پاک می‌کند.

مجاز نقش با کلاس‌های DRF پیاده شده است: ویوهای `/api/driver/...` فقط برای `is_driver` و ویوهای `/api/operator/...` فقط برای `is_operator` باز هستند. کاربر راننده نمی‌تواند بار متصدی دیگری را حذف کند؛ `queryset` متصدی با `request.user` فیلتر می‌شود.

۴.۵ منطق بار و تخصیص

ایجاد بار فقط از مسیر متصدی مجاز است. سریالایزر فیلدهای عنوان، توضیحات، قیمت، وزن، کالا، ماشین، استان‌ها و نشانی‌ها را اعتبارسنجی می‌کند. مختصات جغرافیایی اختیاری است؛ اگر کلاینت نفرستد، سرور می‌تواند بعداً از نشانی پر کند یا خالی بگذارد. وضعیت اولیه `open` است.

فهرست راننده فقط بارهای `open` را برمی‌گرداند و فیلترهای `ostan_maghsad`، `ostan_mabda`، `price_min` و `price_max` را روی `queryset` اعمال می‌کند. پذیرش بار در یک تراکنش انجام می‌شود: قفل ردیف بار، بررسی وضعیت باز بودن، بررسی سازگاری ماشین راننده با ماشین بار، ساخت رکورد تخصیص، و تغییر وضعیت به `assign`. اگر بار پیش‌تر گرفته شده باشد، پاسخ خطا برمی‌گردد.

تکمیل کار دو نقطه پایانی جدا دارد. هر کدام فقط پرچم مربوط به نقش را `True` می‌کند. وقتی هر دو پرچم درست باشند، وضعیت بار `done` می‌شود. این کار با `update` روی مدل تخصیص و در صورت لزوم `save` روی بار، داخل `transaction.atomic` انجام می‌شود تا حالت میانی دیده نشود.

۵.۵ سریالایزرها

DRF برای هر مورد یک سریالایزر خواندن و یک سریالایزر نوشتن دارد. در خواندن بار، نام‌های نمایشی `product_name`، `machine_name`، `ostan_mabda_name`، `operator_name` و در صورت وجود `assign_info` به صورت تو در تو اضافه می‌شوند تا کلاینت مجبور به درخواست اضافه نباشد. در نوشتن، فقط شناسه کالا و ماشین و استان پذیرفته می‌شود. وزن به صورت عدد اعشاری ذخیره می‌شود؛ برای سازگاری با نسخه‌های قدیمی، توضیحات ممکن است پسوند وزن هم داشته باشد.

۶.۵ خطاها و پیام‌ها

پاسخ خطا به صورت JSON با کلید متعارف (`detail` یا `error` یا خطاهای فیلدی سریالایزر) برمی‌گردد. کلاینت این پیام‌ها را با `ApiResponse` به متن قابل نمایش تبدیل می‌کند. کدهای ۴۰۱ برای ژتون نامعتبر، ۴۰۳ برای نقش نادرست، ۴۰۴ برای بار ناموجود و ۴۰۰ برای اعتبارسنجی فرم استفاده می‌شوند.

۷.۵ استقرار سرور

نسخه در حال بهره‌برداری آموزشی روی لیارا با دامنه زیر در دسترس است:

<https://tran-develoop.liara.run>

جنگو در این محیط با WSGI/ASGI پشت پیشکار اجرا می‌شود. فایل پایگاه SQLite روی دیسک پایدار سرویس قرار می‌گیرد. کلیدهای نشان اگر از سرور پروکسی شوند در متغیر محیطی می‌مانند و به کلاینت وب داده نمی‌شوند.

فصل ۶ پایگاه داده و نگاشت شیء-رابطه‌ای

داده‌های سامانه در پایگاه SQLite3 پیش‌فرض جنگو ذخیره می‌شوند و تمام خواندن و نوشتن از مسیر ORM جنگو انجام می‌گیرد؛ در لایه کاربرد دستور SQL خام برای عملیات اصلی بار و حساب نوشته نشده است.

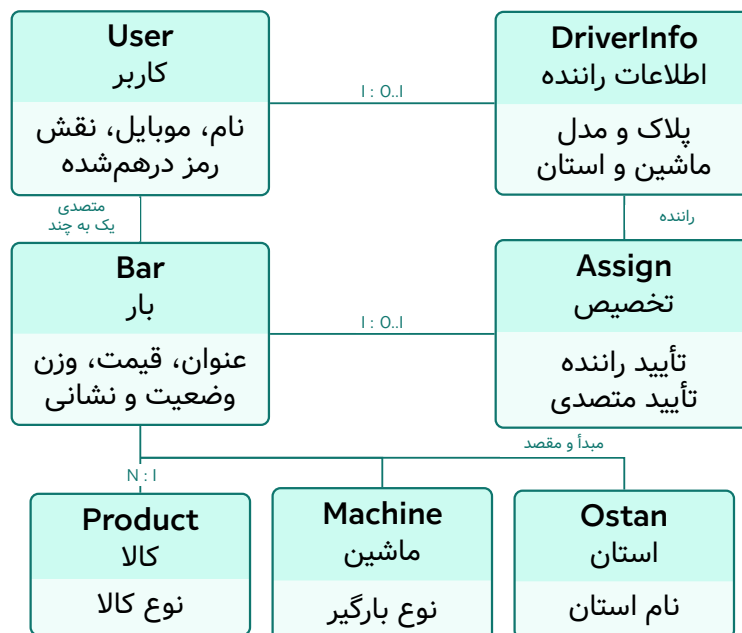
۱.۶ چرا SQLite و ORM

برای پروژه کارشناسی با بار همزمان محدود، SQLite مزایای مشخصی دارد: بدون فرایند جدا، پشتیبان‌گیری با کپی یک فایل، و سازگاری کامل با مهاجرت‌های جنگو. ORM جنگو این انتزاع را می‌دهد که مدل پایتون منبع حقیقت طرح باشد. دستور **makemigrations** تغییر فیلد را به فایل مهاجرت تبدیل می‌کند و **migrate** آن را روی **db.sqlite3** اعمال می‌کند. در نتیجه طرح پایگاه و کد پایتون از هم دور نمی‌افتند.

اگر در آینده تعداد تراکنش‌های همزمان بالا برود، کافی است **DATABASES** به PostgreSQL عوض شود. مدل‌ها، سریالایزرها و ویوها بدون بازنویسی منطق باقی می‌مانند.

۲.۶ مدل‌های دامنه

طرح منطقی مطابق شکل ۱.۶ است.



شکل ۱.۶: نمای موجودیت-رابطه منطقی سامانه

۱.۲.۶ کاربر

کاربر با شماره موبایل یکتا شناسایی می‌شود. رمز عبور فقط به صورت درهم‌شده جنگو ذخیره می‌گردد. دو بولین `is_driver` و `is_operator` نقش را مشخص می‌کنند. نام، نام خانوادگی و کد ملی فیلدهای پروفایل‌اند.

۲.۲.۶ اطلاعات راننده

پروفایل راننده به کاربر و به رکورد ماشین و استان وصل است. پلاک در قالب قابل تجزیه برای ویجت پلاک ایرانی ذخیره می‌شود. مدل خودرو متن آزاد است. ظرفیت تناژ در صورت پشتیبانی سرور در همین پروفایل قابل گسترش است.

۳.۲.۶ بار

جدول بار هسته دامنه است. فیلدهای اصلی در جدول ۱.۶ آمده‌اند. متصدی با کلید خارجی به کاربر محدود می‌شود تا هر متصدی فقط بار خودش را ببیند.

۴.۲.۶ تخصیص

رابطه یک‌به‌یک یا یک‌به‌یک ناقص با بار: هر بار حداکثر یک تخصیص فعال دارد. فیلدهای `confirm_driver` و `confirm_operator` پیشرفت تکمیل را نگه می‌دارند. نام و تلفن راننده می‌تواند از کاربر مرتبط خوانده شود یا در پاسخ سریالایزر تزریق شود.

۵.۲.۶ جداول مرجع

کالا، ماشین و استان فهرست‌های نسبتاً ثابت‌اند. کلاینت آن‌ها را در فرم‌ها به صورت شناسه ارسال می‌کند تا از متن آزاد و ناسازگار جلوگیری شود.

۳.۶ قیدها و صحت داده

در سطح مدل، شماره موبایل یکتا است. وضعیت بار فقط از مجموعه مجاز انتخاب می‌شود. پذیرش بار با `select_for_update` روی ردیف بار از تخصیص همزمان دو راننده جلوگیری می‌کند. حذف بار در حالت تخصیص‌یافته می‌تواند در لایه ویو ممنوع شود تا مأموریت فعال از بین نرود. مهاجرت‌ها تاریخچه طرح را نگه می‌دارند؛ افزودن فیلد `weight` بدون شکستن توضیحات قدیمی نمونه‌ای از تکامل تدریجی طرح است که کلاینت با خواندن هر دو منبع آن را تحمل می‌کند.

۴.۶ نمونه عملیات ORM

به جای نوشتن SQL، ویوها از زنجیره `queryset` استفاده می‌کنند. نمونه مفهومی فهرست بار راننده:

```
Bar.objects.filter(status="open")
.filter(ostan_mabda=ostan_id)
.filter(price__gte=min_price)
.select_related("product", "machine")
.order_by("-created_at")
```

پذیرش در تراکنش:

جدول ۱.۶: مهم‌ترین فیلدهای مدل بار

مشخصات بار		
فیلد مدل	نوع	توضیح
title	متن	عنوان بار
description	متن بلند	شرح بار
price	عدد صحیح	قیمت به ریال
weight	اعشاری	وزن به تن
کلیدهای خارجی		
product	کلید خارجی	نوع کالا
machine	کلید خارجی	نوع بارگیر
مکان		
ostan_mabda	کلید خارجی	استان مبدأ
ostan_maghsad	کلید خارجی	استان مقصد
address_mabda	متن	نشانی مبدأ
address_maghsad	متن	نشانی مقصد
latitude_*	اعشاری	مختصات نقشه، اختیاری
وضعیت و زمان		
status	انتخاب	باز، تخصیص، تحویل یا لغو
created_at	زمان	ایجاد خودکار

```

with transaction.atomic():
    bar = Bar.objects.select_for_update().get(pk=pk)
    Assign.objects.create(bar=bar, driver=request.user)
    bar.status = "assign"

    bar.save(update_fields=["status"])

```

این سبک خوانایی، جلوگیری از تزریق SQL، و هماهنگی با مهاجرت‌ها را هم‌زمان تأمین می‌کند.

فصل ۷ قرارداد وب سرویس ها و امنیت

این فصل قرارداد عملی میان کلاینت فلاتر و سرور جنگو را مستند می کند. آدرس پایه در محیط استقرار آموزشی به صورت زیر است:

<https://tran-deveoop.liara.run>

۱.۷ حساب کاربری

جدول ۱.۷: نقاط پایانی حساب

روش	مسیر	احراز	شرح
POST	/api/accounts/login	خیر	ورود؛ پاسخ کاربر و ژتون
POST	/api/accounts/register/	خیر	ایجاد حساب
POST	/api/accounts/logout	بله	خروج سمت سرور
GET	/api/accounts/profile/	بله	خواندن پروفایل
PUT	/api/accounts/profile/	بله	به روز رسانی نام، پلاک، ماشین
POST	/api/accounts/change-password/	بله	تغییر رمز

بدنه ورود حداقل شامل `phone_number` و `password` است. پاسخ موفق کلیدهای `user` و `tokens.access` دارد. ثبت نام علاوه بر نام و رمز، `password_confirm` و `user_type` (driver) یا `operator` می خواهد. راننده می تواند `machine_id` و `ostan_id` بفرستد.

۲.۷ بار متصدی

جدول ۲.۷: نقاط پایانی بار برای متصدی

روش	مسیر	شرح
GET	/api/operator/bars/	فهرست بارهای متصدی جاری
POST	/api/operator/bars/create/	ایجاد بار
GET	/api/operator/bars/<id>/	جزئیات
PATCH	/api/operator/bars/<id>/update/	ویرایش
DELETE	/api/operator/bars/<id>/delete/	حذف
POST	/api/operator/bars/<id>/complete/	تأیید تکمیل متصدی

بدنهٔ ایجاد/ویرایش با فیلدهای `title`، `description`، `price`، `weight`، `product`، `machine`، `ostan_mabda`، `ostan_maghsad`، `address_mabda` و `address_maghsad` هم‌خوان است. مختصات اختیاری‌اند.

۳.۷ بار راننده

جدول ۳.۷: نقاط پایانی بار برای راننده

روش	مسیر	پرس‌وجو	شرح
GET	<code>/api/driver/bars/</code>	استان، قیمت، صفحه	بارهای باز قابل پذیرش
GET	<code>/api/driver/bars/assigned/</code>	صفحه	مأموریت‌های تخصیص‌یافته
POST	<code>/api/driver/bars/<id>/accept/</code>	—	پذیرش
POST	<code>/api/driver/bars/<id>/complete/</code>	—	تأیید تکمیل راننده

پذیرش و تکمیل بدنه ندارند. منطق روی هویت ژتون و وضعیت فعلی بار است.

۴.۷ دادهٔ مرجع

سه مسیر GET بدون پیچیدگی دامنه:

- `/api/products/`
- `/api/machines/`
- `/api/ostans/`

هر مورد حداقل شناسه و نام دارد تا فرم‌های فلاتر فهرست کشویی بسازند.

۵.۷ امنیت ارتباط

همهٔ مسیرهای پس از ورود با HTTPS و سرآیند `Authorization: Bearer` حفاظت می‌شوند. ژتون در Flutter Secure Storage می‌ماند نه در SharedPreferences. کلاینت وب همان الگو را با محدودیت‌های ذخیره‌سازی مرورگر رعایت می‌کند.

نقش در کلاینت فقط برای هدایت رابط کاربری است؛ تصمیم نهایی مجوز در سرور گرفته می‌شود. اگر راننده مسیر ایجاد بار متصدی را صدا بزند، DRF باید ۴۰۳ بدهد.

رمز عبور هرگز در پاسخ JSON برنمی‌گردد. پلاک و کد ملی فقط برای کاربر مالک پروفایل در `/api/accounts/profile/` GET دیده می‌شود.

مهلت بی‌فعالیتی ۱۵ دقیقه‌ای در کلاینت لایهٔ دفاعی اضافه است و جایگزین انقضای ژتون سرور نمی‌شود.

۶.۷ صفحه‌بندی و سازگاری

پاسخ‌های فهرستی پوششی روی **results** دارند. کلاینت تا وقتی **next** موجود باشد یا **current_page** از **total_pages** کوچک‌تر باشد، صفحه را افزایش می‌دهد. این رفتار برای مأموریت‌های راننده حیاتی است تا صفحه دوم حاوی بارهای **done** از قلم نیفتد. برای وزن، کلاینت هم فیلد عددی جدید و هم پسوند متنی قدیمی را می‌فهمد تا استقرار تدریجی سرور بدون شکستن اپلیکیشن ممکن باشد.

فصل ۸ استقرار، نسخه وب و آزمون

۱.۸ محیط‌های اجرا

سه محیط در پروژه از هم جدا شده‌اند:

۱. توسعه محلی کلاینت: `flutter run` روی اندروید یا کروم با امکان `USE MOCK API=true`.
۲. کلاینت متصل به سرور واقعی: پیش‌فرض ساخت، آدرس لیارا است.
۳. وب پیش‌رونده انتشار: `flutter build web --release` و بسته‌بندی در `dist/logistics-pwa-build`.

سرور جنگو روی لیارا با همان پایگاه SQLite و مهاجرت‌های اعمال‌شده اجرا می‌شود. کلاینت اندروید و PWA به یک قرارداد API وصل‌اند؛ بنابراین ناسازگاری سکویی در منطق بار وجود ندارد.

۲.۸ ساخت نسخه وب پیش‌رونده

ساخت انتشار با راهبرد بدون سرویس‌ورکر پیش‌فرض فلاتر انجام می‌شود تا با سرویس‌ورکر اختصاصی لجستیک رقابت نکند. پس از ساخت:

- `index.html` با پایه / و ثبت سرویس‌ورکر نسخه‌بندی‌شده ذخیره می‌شود؛
- `404.html` رونوشت همان صفحه برای میزبان‌های ایستا است؛
- `htaccess`. مسیرهای عمیق را به `index.html` برمی‌گرداند؛
- پرونده فشرده `logistics-pwa-build.zip` برای آپلود آماده می‌شود.

تغییر نام کش سرویس‌ورکر باعث می‌شود نصب قبلی پورسته کهنه را دور بریزد. فایل‌های دودویی بزرگ مانند `canvaskit.wasm` با کش طولانی و پرونده‌های `index.html` و `main.dart.js` بدون کش اجباری سرو می‌شوند.

۳.۸ نقشه در استقرار وب

در وب، SDK بومی اندروید در دسترس نیست. پل جاوااسکریپت نشان بارگذاری می‌شود و ویجت فلاتر از طریق `dart:js_util` با آن حرف می‌زند. این جداسازی اجازه می‌دهد مسیریابی خیابانی روی PWA هم باقی بماند.

۴.۸ آزمون

سمت کلاینت مجموعه `flutter test` قرارداد را قفل کرده است؛ از جمله:

- مسیرها و میزبان پیش‌فرض در `ApiConfig`؛
- نگاشت پروفایل `is_driver` و `driver_info`؛
- پذیرش وزن فارسی/عربی و فیلد عددی؛

- **POST** خالی برای تکمیل هر دو نقش؛
 - پیمایش چندصفحهٔ مأموریت‌های تخصیص‌یافته؛
 - سازگاری ایجاد بار وقتی سرور هنوز **weight** را نپذیرد.
- سمت سرور با آزمون‌های مدل و API جنگو (کلاینت آزمون DRF) ایجاد کاربر، صدور ژتون، ممنوعیت نقش متقابل، و تراکنش پذیرش همزمان قابل راستی‌آزمایی است. اسکریپت `scripts/test_transport_api.sh` برای تست دستی زندهٔ میزبان لیارا استفاده می‌شود.

۵.۸ موارد ناتمام آگاهانه

اعلان فشاری واقعی هنوز به نقطهٔ پایانی ثبت دستگاه وصل نیست و در کلاینت فقط جای‌نگهدار دارد. فهرست رانندگان نزدیک در حالت زنده به علت نبود نقطهٔ پایانی سرور خالی برمی‌گردد و فقط در حالت ساختگی داده دارد. این موارد در فصل بعد به عنوان کار آینده آمده‌اند.

فصل ۹ نتیجه‌گیری و پیشنهادها

۱.۹ جمع‌بندی

سامانه لجستیک یک زنجیره کامل از ثبت بار تا تحویل را برای حمل جاده‌ای پیاده کرده است. در سمت کاربر، فلاتر تجربه دوزبانه، امن و چندسکویی (اندروید و وب پیش‌رونده) فراهم می‌کند و نقشه نشان مسیر واقعی را به راننده می‌دهد. در سمت سرور، جنگو و DRF قرارداد REST نقش‌محور را با ژتون JWT عرضه می‌کنند و ORM جنگو همه تغییرات را روی SQLite3 پایدار می‌سازد.

تقسیم کار روشن بوده است: **حانیه نباتی** کل لایه ارائه و اتصال کلاینت، و **مهیار عامری** خدمت، مجوزها و پایگاه داده را بر عهده داشته‌اند. هدایت علمی پروژه با **دکتر سید مرتضی بابامیر** انجام شده است.

دستاوردهای فنی مهم، یکسان بودن منبع حقیقت وضعیت بار در سرور و ترجمه دقیق آن در کلاینت است؛ به ویژه تکمیل دو تأیید که جلوی بستن یک‌طرفه مأموریت را می‌گیرد.

۲.۹ آموخته‌ها

- قرارداد API باید پیش از جزئیات ویجت تثبیت شود؛ نگاشت متمرکز در کلاینت تغییر فیلد سرور را تحمل‌پذیر کرد.
- نقش فقط در رابط کاربری کافی نیست؛ بدون مجوز DRF هر مسیر قابل جعل است.
- SQLite برای شروع درست است اگر مهاجرت‌ها منظم باشند و مسیر ارتقا به موتور دیگر باز بماند.
- وب پیش‌رونده بدون نسخه‌بندی کش، به‌روزرسانی را از کاربر دریغ می‌کند.

۳.۹ پیشنهاد کارهای آینده

۱. اتصال FCM و نقطه پایانی ثبت دستگاه در جنگو برای اعلام بار جدید و تغییر وضعیت.
۲. ارائه موقعیت لحظه‌ای راننده به متصدی (با رضایت و کنترل حریم خصوصی) و فهرست رانندگان نزدیک واقعی.
۳. نقطه تازه‌سازی ژتون و فهرست سیاه خروج.
۴. جایگزینی SQLite با PostgreSQL در تولید و پشتیبان‌گیری دوره‌ای.
۵. موتور قیمت‌گذاری سمت سرور بر اساس مسافت نشان و نوع ماشین.
۶. پرداخت و تسویه، و تاریخچه مالی مأموریت.
۷. آزمون بار و نظارت تأخیر API روی لیارا.

۴.۹ سخن پایانی

هدف آموزشی پروژه، تمرین مهندسی نرم‌افزار سرتاسری بود: مدل دامنه، واسط پایدار، کلاینت چندسکویی، و استقرار قابل نمایش. سامانه در وضعیت فعلی برای سناریوی آموزشی و بهره‌برداری محدود عملیاتی آماده است و مسیر رشد آن بدون شکستن قرارداد فعلی قابل تعریف است.

کتابنامه

- [۱] Django Software Foundation. Django documentation.
<https://docs.djangoproject.com/>
- [۲] Encode. Django REST framework.
<https://www.django-rest-framework.org/>
- [۳] Open Worldwide Application Security Project. JSON Web Token.
<https://jwt.io/>
- [۴] Jazzband. Simple JWT for Django REST framework.
<https://django-rest-framework-simplejwt.readthedocs.io/>
- [۵] Hipp, D. R. SQLite documentation.
<https://www.sqlite.org/docs.html>
- [۶] Google. Flutter documentation.
<https://docs.flutter.dev/>
- [۷] Neshan Platform. Maps and routing documentation.
<https://platform.neshan.org/>
- [۸] Mozilla Developer Network. Progressive web apps.
https://developer.mozilla.org/docs/Web/Progressive_web_apps

چکیده انگلیسی

Logistics: A Smart Road Freight Management System

Hanieh Nabati (Frontend) Mahyar Ameri (Backend & Database)

Supervisor: Dr. Seyed Morteza Babamir

University of Kashan, Faculty of Electrical and Computer Engineering

Logistics is a client-server platform that connects road-freight *drivers* and *transport operators*. Operators register cargo with product type, vehicle class, weight, price and origin/destination. Drivers browse matching open cargos, receive GPS-based nearby suggestions, accept a job, navigate street-level routes with Neshan, and complete the mission only after dual confirmation. The mobile/web client is built with Flutter (including a Progressive Web App). The backend is implemented in Django and Django REST Framework, persists data with the Django ORM on SQLite3, and authenticates clients with JWT. The system is deployed at <https://tran-develoop.liara.run>.

Keywords: road freight, logistics, Flutter, Django, DRF, SQLite, JWT, PWA, Neshan routing.